

Kriya — What We've Built (Feature List)

A plain-English tour of everything Kriya can do today, grouped by category.

Nothing here is jargon for its own sake — where a technical term is needed, it's explained.

The one-line idea

Kriya lets you build apps where **AI agents are real users**, not bolt-ons. The agent drives the app the same way a person does — by calling **typed actions** (clear, named commands with defined inputs), instead of reading the screen or scraping the page. The same handler serves a human clicking a button and an agent calling a tool. Safety (permissions, approvals, audit, budgets, memory) is built into the foundation, not added later.

Status key:  Ready ·  Partial (works, more coming) ·  Paid tier

1. Building blocks for developers

The toolkit you use to make your app's features available to an agent.

-  **Register an action** — turn any app feature into a named command an agent can call.
-  **Typed inputs with checking** — each action declares its inputs (types, required-ness, allowed values). Bad input is caught automatically before anything runs.
-  **Auto-generated tool descriptions** — Kriya produces standard schemas, so any AI client understands your actions with no extra work.
-  **Actions can call other actions** — build a big action out of smaller ones; each step still goes through the same safety checks. (Loops and runaway nesting are blocked.)
-  **Wrap functions you already have** (`wrapAction`) — point Kriya at an existing function and it becomes a governed agent action. No rewrite.
-  **Auto-converter (codemod)** — a command that scans your existing code and writes the

wrapper boilerplate for you.

2. The agent engine

The part that actually runs the agent loop.

-  **Step-by-step agent loop** — the agent thinks, acts, sees the result, repeats.
-  **Swappable AI brains** — works with a built-in deterministic planner (no AI key needed,

great for demos/tests), local models via **Ollama**, the **Anthropic API**, and **Claude CLI**.

-  **Runs in its own separate process** — the agent runs apart from your app's main window,

so the safety layer can't be tampered with by the page.

-  **Pause-and-step mode** — step through the agent one decision at a time and watch what it

does, like a debugger.

-  **Resume after a crash** — if a run is interrupted (even right in the middle of waiting for

a human approval), it can pick up where it left off instead of starting over.

-  More AI backends (OpenAI), automatic retries, and "this is too hard, escalate" fallback

are still to come.

3. Safety & control

The guardrails — this is the core of what makes Kriya different.

-  **Permission rules** — a simple policy file decides what's allowed. Default is **deny**:

nothing runs unless you say it can.

-  **Human approval for risky actions** — the agent pauses and waits for a person to approve

or deny. If no one responds in time, it's automatically denied (safe by default).

-  **Approval prompts that fit the setup** — approve via a popup in the app, a terminal

prompt, or a **native Mac dialog box** (handy when the agent runs under another tool).

-  **Budgets & rate limits** — cap how many actions run per minute (live today); a per-hour

API-call cap is still to come.

-  **Policy warnings** — Kriya flags risky policies for you (e.g. "this allows everything",

"a delete action has no approval required") the first time you run.

4. Audit & proof

A trustworthy, tamper-evident record of everything the agent did.

-  **Signed receipts** — every action that runs gets a cryptographically signed record. Blocked actions are never signed (the log only attests to what actually happened).
-  **Who-did-what** — each receipt records the agent *and* the operator behind the action, signed inside the record so it can't be quietly altered.
-  **Offline verifier** — a standalone tool that checks the receipts are genuine, with no internet and no trust in us required.
-  **On-device guarantee** — turn on a sealed "nothing leaves this machine" mode; Kriya refuses any AI backend that would send data out and signs a receipt proving it stayed local. (Built for regulated, privacy-sensitive workplaces.)

5. Memory

So the agent learns across runs instead of forgetting everything each time.

-  **Persistent history** — every action across every run is saved to a local database.
-  **Recall into the agent's thinking** — past runs are fed back into the agent's prompt, so earlier work informs new decisions.
-  **Browse the past** — query recent runs newest-first; see the count at the start of a run.
-  Smarter recall (similarity search / embeddings) and full state snapshots are still to come.

6. Plugging into AI tools

Different ways an external agent or app can connect to Kriya.

-  **MCP server mode** — expose your app's actions over the standard agent protocol so tools like **Claude Desktop** or **Cursor** can drive your app — with every call still routed through the full safety layer.

-  **Sidecar for Electron / Node apps** — run the governed agent engine as a companion process

for non-Tauri apps, talking over a simple text protocol.

-  **Python SDK** — `pip install kriya`. The same register/wrap/validate toolkit and host driver, now in Python (opens up CAD, data/ML, scientific, and finance tools).
-  More languages planned next (.NET, then Java/Kotlin), demand-driven.

7. Developer experience

Tools that make building and debugging pleasant.

-  **One-command starter** — `npm create kriya-app` scaffolds a complete working app (UI + agent host + safety layer + locked dependencies) with a README.
-  **Inspector dashboard** — a visual panel with a searchable, filterable log, per-step detail, one-click export, a **memory browser** with step-through replay, and the pause/step controls.
-  **Action dump CLI** — list all registered actions as JSON.
-  Tutorials, an examples gallery, and tracing/CI checks are still to come.

8. Example apps & demos

Real, runnable proof — same engine, different domains.

-  **Note app** — the reference app (create/edit/delete notes), with "Resume last task".
-  **Task manager** — a second app on the same engine, including approval-gated actions.
-   **Actual Budget bolt-on (flagship)** — governed agent control of a real, local-only

finance app in about **37 lines** of glue, no rewrite. Deletes and account-closes require human approval; everything is signed and audited.

-  **On-device guarantee demo** — shows the "nothing leaves the machine" mode in action.
-  **Identity demo** — shows who-did-what stamped into each receipt.
-  **Node & Python host examples** — drive the governed runtime from plain Node or Python.

9. Published & available

-  **npm packages** — the core SDK, the sidecar, the inspector, and the app scaffolder.
-  **Rust crate** — the agent host engine.
-  **Python package** — `pip install kriya`.

10. Paid tier (separate private product)

Built on the same audit/approval/budget foundation, sold to enterprises.

-   **Compliance evidence export** — turn the audit log into ready-made evidence bundles for standards like SOC 2, ISO 42001, and the EU AI Act.
-   **Governance dashboard** — a cross-app, cross-agent audit viewer with in-app policy editing, approval routing, and budget controls (in progress).

Source of truth: `docs/PRODUCT_GAPS.md` and `docs/ROADMAP.md`. This is a plain-English summary — those docs hold the exact status and commit history.